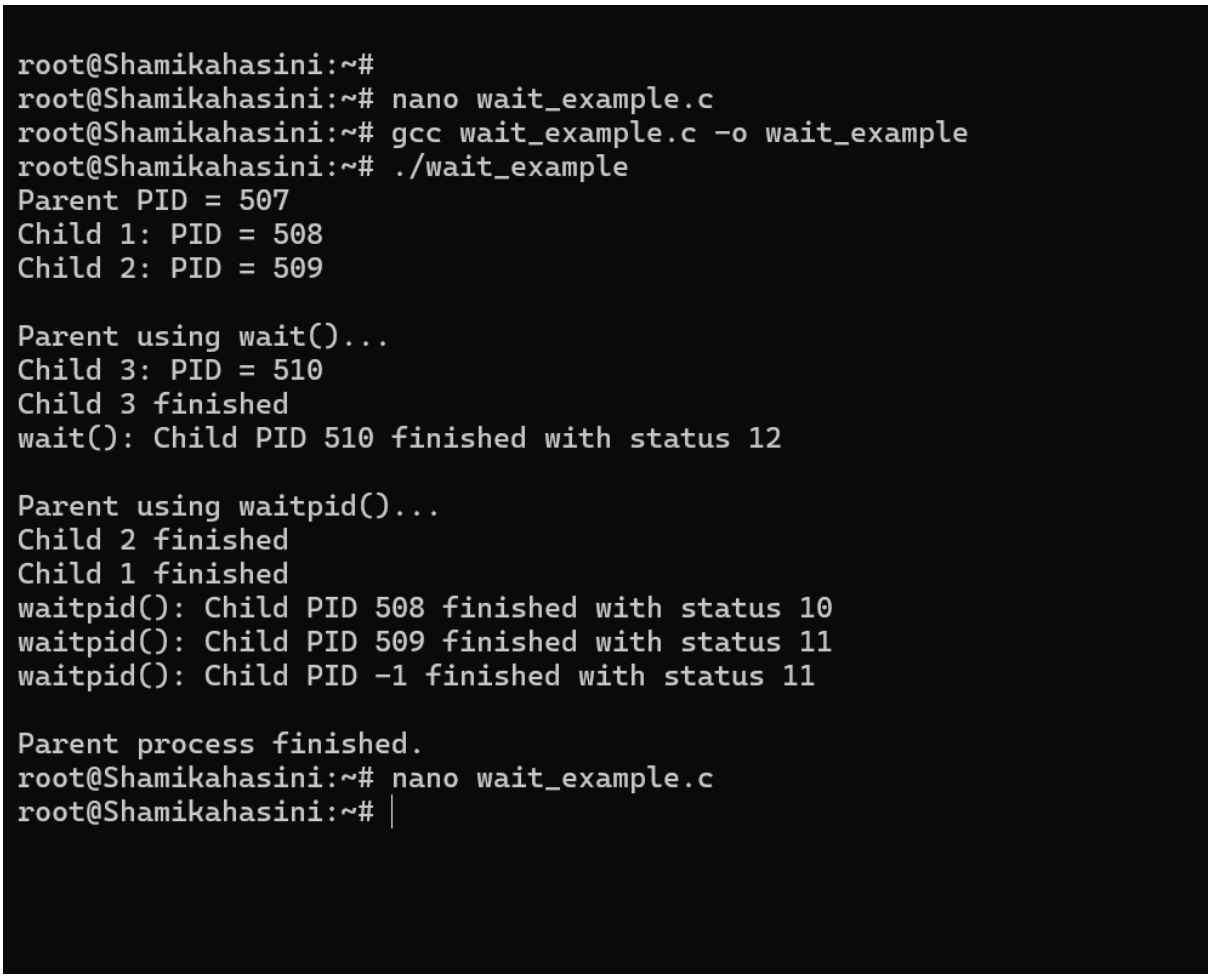
Week-4 practical

Task-1

Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
  GNU nano 7.2                                                    wait_e
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t children[3];
    int status;

    printf("Parent PID = %d\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < 3; i++)
    {
        children[i] = fork();

        if (children[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (children[i] == 0)
        {
            printf("Child %d: PID = %d\n", i + 1, getpid());

            sleep(3 - i);

            printf("Child %d finished\n", i + 1);
            exit(10 + i);
        }
    }

    // wait() waits for any one child
    printf("\nParent using wait()...\n");

    pid_t pid = wait(&status);

    if (WIFEXITED(status))
```

```
            exit(10 + i);
        }
    }

    // wait() waits for any one child
    printf("\nParent using wait()...\n");

    pid_t pid = wait(&status);

    if (WIFEXITED(status))
    {
        printf("wait(): Child PID %d finished with status %d\n",
               pid, WEXITSTATUS(status));
    }

    // waitpid() waits for specific children
    printf("\nParent using waitpid()...\n");

    for (int i = 0; i < 3; i++)
    {
        pid = waitpid(children[i], &status, 0);

        if (WIFEXITED(status))
        {
            printf("waitpid(): Child PID %d finished with status %d\n",
                   pid, WEXITSTATUS(status));
        }
    }

    printf("\nParent process finished.\n");

    return 0;
}
```



Observation

The parent process creates three child processes using the `fork()` system call. The child processes execute independently and terminate at different times. The `wait()` function allows the parent process to wait for any one child process to complete, while `waitpid()` allows the parent to wait for a specific child process using its process ID. Thus, both functions are used to synchronize the parent and child processes.

Result

The C program was successfully compiled and executed in Ubuntu. The program demonstrates process synchronization using `wait()` and `waitpid()`. It was observed that `wait()` waits for any child process to terminate, whereas `waitpid()` provides control to wait for a particular child process.

Task-2

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child is exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent is sleeping...\n");

        sleep(30);
    }

    return 0;
}
```

```
root@Shamikahasini:~# nano zombie.c
root@Shamikahasini:~# gcc zombie.c -o zombie
root@Shamikahasini:~# ./zombie
Parent process: PID = 578
Child PID = 579
Child process: PID = 579
Child is exiting...
Parent collected child process.
root@Shamikahasini:~# ps -el | grep Z
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
```

Observation:

The program successfully created a parent and child process using fork(). The child process terminated while the parent process continued executing and slept for 30 seconds. Since the parent did not call wait() to collect the child's termination status, the terminated child temporarily remained in the process table as a zombie process (Z state). After the parent process completed, the zombie was removed. The program was then modified to use proper synchronization with wait(), allowing the parent to collect the child's termination status and preventing the creation of a zombie process.

Result:

A zombie process was successfully created and observed, and proper synchronization using wait() was used to eliminate it.